TUGAS Hands On 1: MODEL EXPERIMENTATION & TRACKING

Nama : Lutfia Aisyah Putri
NIM : 123450074
Mata Kuliah : Machine Learning Operations (MLOps) RB

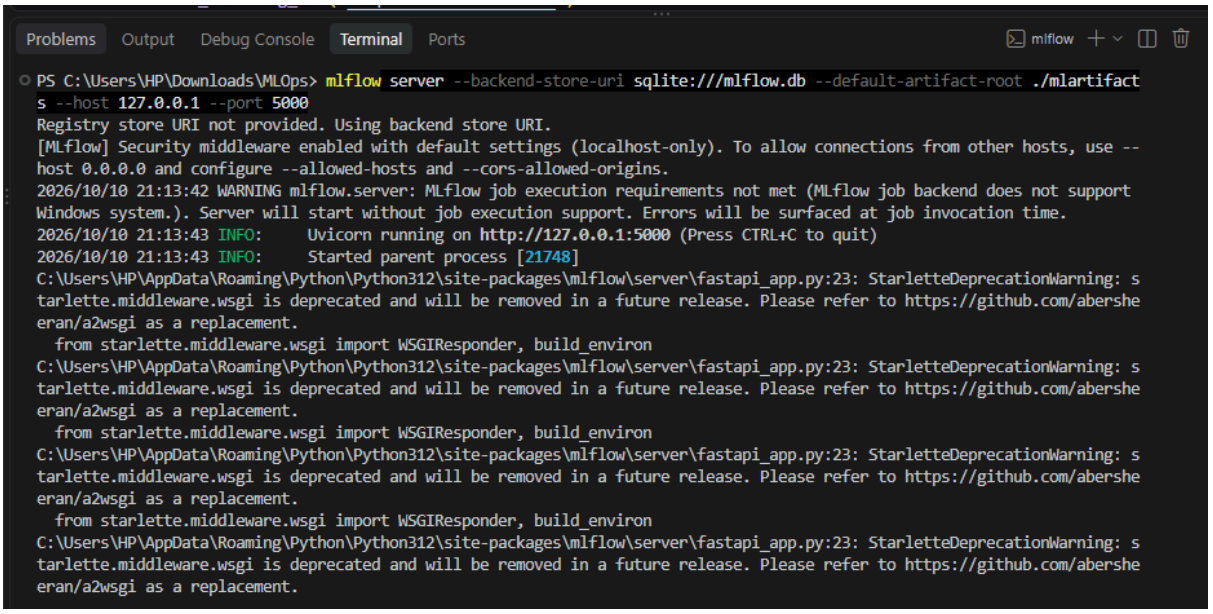
1. Tujuan dan Hipotesis

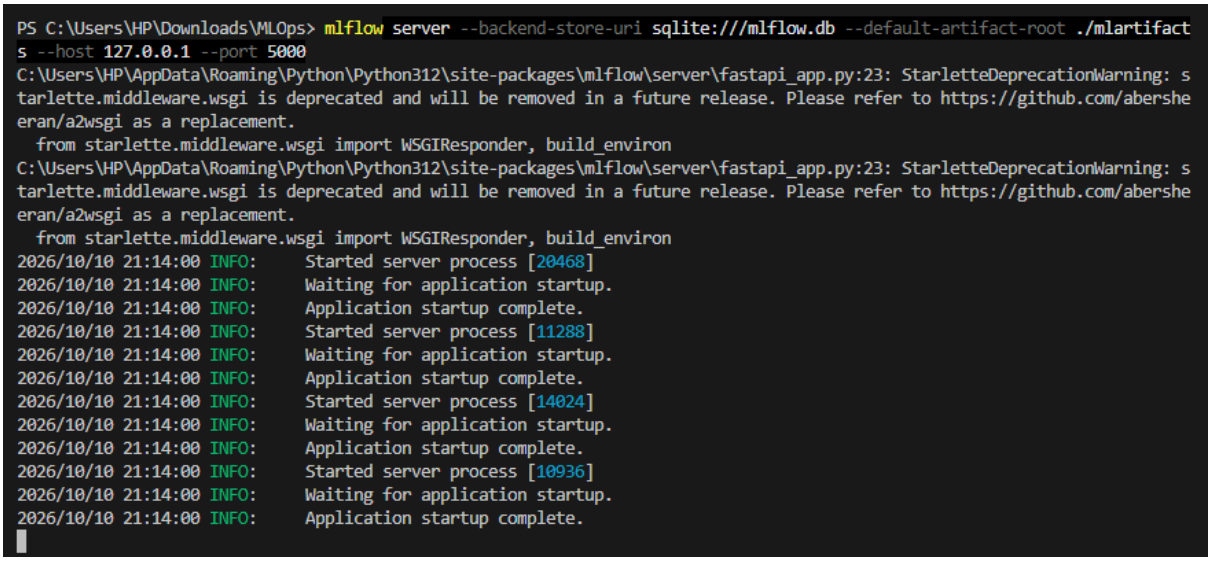
- Tujuan: Membangun alur eksperimen machine learning menggunakan MLflow untuk melacak parameter, metrik evaluasi, dan model secara terpusat.
- Hipotesis Eksperimen: Terdapat perbedaan kemudahan dan kelengkapan logging antara metode Manual Logging (menggunakan Decision Tree) dibandingkan dengan metode Autologging (menggunakan Logistic Regression).

2. Skema Eksperimen Pada praktikum ini, dataset dan algoritma telah disesuaikan (berbeda dari contoh di kelas):

- Dataset: Wine Quality Dataset (Klasifikasi multi-kelas untuk menentukan kualitas jenis anggur).
- Algoritma 1 (Manual Logging): Decision Tree Classifier (Mencatat parameter dan metrik seperti *accuracy* dan *f1_macro* secara manual).
- Algoritma 2 (Autologging): Logistic Regression (Menggunakan fungsi mlflow.sklearn.autolog() untuk mencatat keseluruhan parameter dan metrik secara otomatis)

3. Persiapan: Inisialisasi MLflow Tracking Server





Sebelum mengeksekusi skrip eksperimen berbasis Python, sebuah *local tracking server* MLflow harus diaktifkan terlebih dahulu melalui antarmuka terminal. Gambar log di atas membuktikan bahwa peladen (*server*) telah berhasil diinisialisasi dan berjalan. MLflow dikonfigurasi untuk menggunakan *backend* basis data SQLite (mlflow.db) guna merekam metadata eksperimen, dan menyiapkan direktori ./mlartifacts untuk menyimpan *file* model fisik. Baris informasi *Uvicorn running on* http://127.0.0.1:5000 secara definitif mengonfirmasi bahwa *tracking server* telah berstatus aktif dan siap melacak (*tracking*) siklus eksperimen model *Machine Learning* yang akan dieksekusi.

4. Skrip Kode Eksperimen

a. Kode Manual Logging (Decision Tree)

```
import mlflow
import mlflow.sklearn
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, f1_score

# Hubungkan langsung ke database lokal
mlflow.set_tracking_uri("http://127.0.0.1:5000")
mlflow.set_experiment("Wine-Quality-Classification")

X, y = load_wine(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

```
params = {"max_depth": 5, "random_state": 42, "criterion":
"entropy"}

with mlflow.start_run(run_name="DecisionTree_Manual"):
    mlflow.set_tags({"stage": "eksperimen_awal", "metode":
"manual"})
    mlflow.log_params(params)

    model = DecisionTreeClassifier(**params)
    model.fit(X_train, y_train)

    y_pred = model.predict(X_test)
    acc = accuracy_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred, average="macro")

    mlflow.log_metric("accuracy", acc)
    mlflow.log_metric("f1_macro", f1)

    # Mematikan skops (memakai cloudpickle) agar 100% aman dari
error Windows
    mlflow.sklearn.log_model(
        model,
        "model_decision_tree",

serialization_format=mlflow.sklearn.SERIALIZATION_FORMAT_CLOUDPIC
KLE
    )

    print(f"[Manual Log] Run berhasil! Accuracy: {acc:.4f},
F1-Score: {f1:.4f}")
```

Eksperimen pertama menerapkan pendekatan manual logging untuk melacak siklus pelatihan model Decision Tree Classifier menggunakan Application Programming Interface (API) dari MLflow. Setelah melakukan inisialisasi koneksi ke peladen (tracking server) lokal dan membagi dataset Wine, seluruh proses pelacakan dienkapsulasi di dalam sebuah context manager mlflow.start_run(). Pada pendekatan ini, pelacakan dilakukan secara preskriptif di mana developer harus mendefinisikan instruksi pencatatan secara eksplisit. Fungsi mlflow.log_params() digunakan untuk melakukan logging konfigurasi hyperparameter (seperti max_depth dan criterion). Setelah melalui fase inferensi pada data uji, fungsi mlflow.log_metric() dipanggil secara manual untuk merekam kuantifikasi performa evaluasi, yakni accuracy dan f1_macro. Rangkaian instruksi ini kemudian diakhiri dengan proses serialisasi dan penyimpanan artifak model melalui modul mlflow.sklearn.log_model(), yang

mengandalkan format cloudpickle untuk persistensi model secara aman di dalam repositori MLflow.

b. Kode Autologging (Logistic Regression)

```
import mlflow
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

# Hubungkan langsung ke database lokal
mlflow.set_tracking_uri("http://127.0.0.1:5000")
mlflow.set_experiment("Wine-Quality-Classification")

# Aktifkan Autologging
mlflow.sklearn.autolog()

X, y = load_wine(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

with mlflow.start_run(run_name="LogisticRegression_Auto"):
    # Aman dari error karena algoritma bukan berbasis Tree
    model = LogisticRegression(max_iter=10000, random_state=42)
    model.fit(X_train, y_train)

    print("[Auto Log] Run autologging berhasil!")
```

Eksperimen kedua mengeksploitasi kapabilitas autologging dari MLflow untuk mereduksi kompleksitas kode (boilerplate) pada pelatihan algoritma Logistic Regression. Titik fokal dari skrip ini terletak pada pemanggilan fungsi integrasi mlflow.sklearn.autolog() sebelum eksekusi context manager. Melalui fungsi tersebut, MLflow secara internal melakukan hooking atau injeksi otomatis pada metode fit() milik pustaka Scikit-Learn. Alhasil, ketika prosedur pelatihan model.fit() dieksekusi, MLflow secara dinamis mengekstraksi seluruh hyperparameter bawaan maupun kustom dari algoritma klasifikasi, mengkalkulasi metrik pelatihan berkesinambungan (training loss dan accuracy score), serta merekam spesifikasi arsitektur model beserta struktur masukannya (signature) secara komprehensif. Pendekatan ini memungkinkan otomasi pelacakan eksperimen secara ekstensif (end-to-end) tanpa memerlukan intervensi deklarasi logging metrik secara eksplisit di dalam blok kode pengembang.

5. Hasil di MLflow UI (Compare View)

Run details

Run ID:	fd72eeb4b93144cfbe1cf11c38b62b48	2954edfb4ad04c15a43a1fed6a858a52
Run Name:	LogisticRegression_Auto	DecisionTree_Manual
Start Time:	10/08/2026, 09:59:12 PM	10/08/2026, 09:58:32 PM
End Time:	10/08/2026, 09:59:40 PM	10/08/2026, 09:58:58 PM
Duration:	28.5s	25.2s

Gambar di atas menampilkan informasi dasar dari kedua eksperimen yang sedang dibandingkan. Tabel ini mengonfirmasi bahwa kolom sebelah kiri mewakili eksperimen Logistic Regression (metode Autologging), sedangkan kolom sebelah kanan mewakili Decision Tree (metode Manual Logging).

Selain nama eksperimen, MLflow juga terbukti sukses merekam metadata run secara otomatis dan akurat, mulai dari Run ID unik, waktu dimulainya training (Start Time), waktu selesai (End Time), hingga total durasi pelatihannya (Duration). Hal ini membuktikan bahwa pelacakan siklus hidup model (MLOps) berjalan dengan sempurna.

Parameters

Show diff only

C	1.0	
class_weight	None	
criterion		entropy
dual	False	
fit_intercept	True	
intercept_scaling	1	
l1_ratio	None	
max_depth		5
max_iter	10000	
multi_class	deprecated	

n_jobs	None	
penalty	l2	
random_state	42	42
solver	lbfgs	
tol	0.0001	
verbose	0	
warm_start	False	

Gambar di atas menunjukkan perbandingan parameter antara model Logistic Regression (kiri) yang menggunakan metode autologging dan Decision Tree (kanan) yang menggunakan metode manual logging.

Dapat dilihat dengan jelas bahwa fitur mlflow.autolog() pada model sebelah kiri secara otomatis menyedot dan mencatat seluruh parameter bawaan (default hyperparameters) dari Scikit-Learn (seperti C=1.0, max_iter=10000, solver=lbfgs, dan lain-lain) tanpa perlu kita tuliskan kodenya satu per satu.

Sebaliknya, pada model sebelah kanan (manual logging), MLflow hanya mencatat parameter yang benar-benar kita definisikan secara eksplisit di dalam kode (yaitu criterion=entropy, max_depth=5, dan random_state=42), sementara kolom lainnya dibiarkan kosong.

Metrics

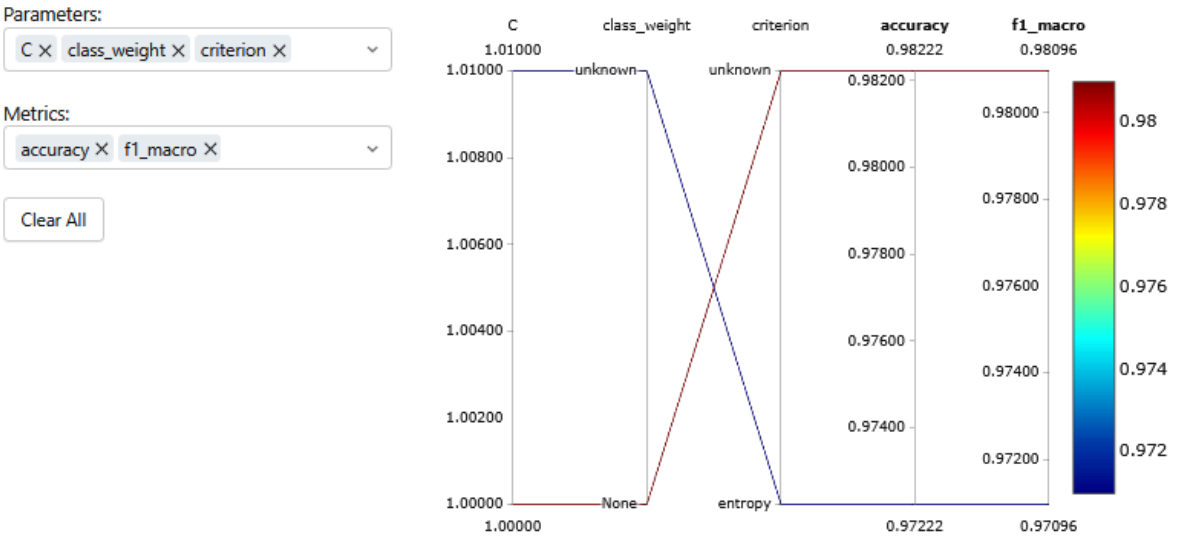
Show diff only

f1_macro	NaN	0.972
training_accuracy_score	NaN	0.971
training_f1_score	0.993	NaN
training_log_loss	0.993	NaN
training_precision_score	0.035	NaN
training_recall_score	0.993	NaN
	0.993	NaN
training_recall_score	1	NaN
training_roc_auc	0.993	NaN
training_score		

Pola yang sama juga terlihat pada tabel Metrics. Model sebelah kiri (autologging) secara otomatis menghitung dan mencatat berbagai macam metrik

pelatihannya (seperti training_accuracy_score, training_f1_score, training_log_loss, dll).

Sementara itu, model sebelah kanan (manual logging) hanya mencatat dua metrik yang kita hitung dan kita perintahkan secara manual melalui kode mlflow.log_metric(), yaitu accuracy (0.971) dan f1_macro (0.972). Hal ini membuktikan fleksibilitas MLflow di mana kita bisa memilih untuk merekam segala sesuatu secara otomatis yang hemat waktu, atau merekamnya secara manual untuk metrik-metrik spesifik yang kita butuhkan.



Gambar grafik di atas memetakan pengaruh spesifik dari hyperparameter terhadap hasil akhir metrik evaluasi model. Berdasarkan garis yang terbentuk, dapat dilihat perbandingan hasil secara langsung:

1. Model Logistic Regression (Garis Atas/Merah): Eksperimen yang menggunakan parameter C = 1.0 terbukti menghasilkan performa paling optimal di antara keduanya, dengan tingkat akurasi (accuracy) mencapai 0.982 (98.2%) dan skor f1_macro sebesar 0.981.
2. Model Decision Tree (Garis Bawah/Biru): Eksperimen yang menggunakan parameter criterion = entropy (dan max_depth = 5) menghasilkan performa yang sedikit lebih rendah, ditunjukkan dengan skor akurasi sebesar 0.972 (97.2%) dan skor f1_macro sebesar 0.971.

Untuk kasus klasifikasi kualitas jenis anggur (Wine Quality) pada rentang data ini, algoritma Logistic Regression memberikan hasil prediksi yang lebih unggul secara keseluruhan dibandingkan algoritma Decision Tree.

6. Kesimpulan

Integrasi MLflow terbukti sangat esensial dalam menjamin integritas dan reprodusibilitas siklus pelatihan model. Analisis komparatif melalui antarmuka *Compare View* dan *Parallel Coordinates Plot* memvalidasi secara objektif bahwa model *Logistic Regression* menghasilkan performa klasifikasi yang lebih superior dibandingkan *Decision Tree* pada dataset *Wine Quality*. Di samping itu, pengujian ini menyoroti perbedaan signifikan pada arsitektur pencatatan (*tracking*), di mana pendekatan *Autologging* menawarkan efisiensi tinggi melalui ekstraksi komprehensif secara otomatis terhadap seluruh *hyperparameter* dan metrik pelatihan, sementara metode *Manual Logging* mengakomodasi kontrol penuh dan granularitas bagi pengembang untuk merekam metrik evaluasi secara spesifik. Secara keseluruhan, penerapan kerangka kerja MLflow ini sukses mendokumentasikan konteks model beserta dependensinya, sehingga memastikan setiap siklus eksperimen dapat diaudit, dibandingkan, dan direkonstruksi secara presisi tanpa khawatir akan hilangnya informasi eksperimental.